

Error Log

Noorinder

1. Missing Override Annotation

- **Error:** The `getDetails()` method in subclasses didn't have the `@Override` annotation, causing potential confusion about inheritance.
- **Cause:** The annotation was omitted during implementation.
- **Fix:** Added the `@Override` annotation to clarify the inheritance relationship.

Before Fix:

```
public String getDetails() {  
    return super.toString() + ", Department: " + departmentName;  
}
```

After Fix:

```
@Override  
public String getDetails() {  
    return super.toString() + ", Department: " + departmentName;  
}
```

2. Incorrect Initialization of Salary

- **Error:** Faculty salary was set to zero because the constructor didn't pass the salary to the parent class.
- **Cause:** The constructor omitted the salary parameter.
- **Fix:** Updated the constructor to include salary and pass it to the parent class.

Before Fix:

```
public Faculty(String name, String departmentName) { // Missing salary parameter  
    super(name, 0); // Salary was set to 0 by default  
    this.departmentName = departmentName;  
}
```

After Fix:

```

public Faculty(String name, double salary, String departmentName) { // Salary parameter added
    super(name, salary); // Passes salary to UEmployee constructor
    this.departmentName = departmentName;
}

```

3. Invalid Employee Selection

- **Error:** Entering an invalid employee number caused a `NullPointerException` or incorrect behavior.
- **Cause:** The program didn't handle invalid input in the `switch` statement.
- **Fix:** Added a `default` case to handle invalid inputs.

Before Fix:

```

switch (empNum) {
    case 1:
        employee = faculty;
        break;
    case 2:
        employee = staff;
        break;
    // No handling for invalid inputs
}

```

After Fix:

```

switch (empNum) {
    case 1:
        employee = faculty;
        break;
    case 2:
        employee = staff;
        break;
    default:
        System.out.println("Invalid Employee number!");
        continue; // Re-prompt the user
}

```

4. Negative Salaries

- **Error:** The `setSalary()` method allowed negative values to be assigned to salary.

- **Cause:** No validation was implemented in the method.
- **Fix:** Added a validation check to ensure only non-negative salaries are accepted.

Before Fix:

```
public void setSalary(double salary) {
    this.salary = salary; // No validation for negative values
}
```

After Fix:

```
public void setSalary(double salary) {
    if (salary >= 0) {
        this.salary = salary; // Assign valid salaries
    } else {
        throw new IllegalArgumentException("Salary cannot be negative.");
    }
}
```

5. Lack of Quit Option

- **Error:** The program didn't provide a clear way for users to exit, causing confusion.
- **Cause:** The menu lacked an explicit quit option.
- **Fix:** Added a **Q** option for quitting the program.

Before Fix:

```
do {
    System.out.println("(E)mployee Details");
    action = input.next();
    // No clear quit option
} while (true);
```

After Fix:

```
do {
    System.out.println("(E)mployee Details | (Q)uit");
    action = input.next();
} while (!action.equalsIgnoreCase("Q"));
System.out.println("Exiting program.");
```